Lab Manual: Analysis and Design of Algorithms

Dr. Mukesh Kumar
Associate Professor
Department of Engineering & Technology
Gurugram University, Gurugram, Haryana

April 13, 2026

Objective

This laboratory manual covers 15 comprehensive experiments in analysis and design of algorithms, organised into 4 modules covering sorting, searching, divide-and-conquer, dynamic programming, greedy algorithms, and graph algorithms. Students will implement classical algorithms, analyse their time complexity, and trace execution on sample inputs.

Contents

1 Experiment 1: Bubble Sort **6**
1.1 Aim 6
1.2 Theory 6
1.3 Algorithm 6
1.4 Program Code 6
1.5 Sample Output 7
1.6 Viva Questions 7

2 Experiment 2: Merge Sort **9**
2.1 Aim 9
2.2 Theory 9
2.3 Algorithm 9
2.4 Program Code 9
2.5 Sample Output 10
2.6 Viva Questions 10

3 Experiment 3: Quick Sort **11**
3.1 Aim 11
3.2 Theory 11
3.3 Algorithm 11
3.4 Program Code 11
3.5 Sample Output 12
3.6 Viva Questions 12

4 Experiment 4: Heap Sort **14**
4.1 Aim 14
4.2 Theory 14
4.3 Algorithm 14
4.4 Program Code 14
4.5 Sample Output 15
4.6 Viva Questions 15

5 Experiment 5: Binary Search 17
5.1 Aim 17
5.2 Theory 17
5.3 Algorithm 17
5.4 Program Code 17
5.5 Sample Output 18
5.6 Viva Questions 18

6 Experiment 6: Strassen Matrix Multiplication 20
6.1 Aim 20
6.2 Theory 20
6.3 Algorithm 20
6.4 Program Code 20
6.5 Sample Output 22
6.6 Viva Questions 22

7 Experiment 7: Maximum Subarray Problem 23
7.1 Aim 23
7.2 Theory 23
7.3 Algorithm 23
7.4 Program Code 23
7.5 Sample Output 24
7.6 Viva Questions 24

8 Experiment 8: Closest Pair of Points 25
8.1 Aim 25
8.2 Theory 25
8.3 Algorithm 25
8.4 Program Code 25
8.5 Sample Output 26
8.6 Viva Questions 26

9 Experiment 9: 0/1 Knapsack Problem 28
9.1 Aim 28
9.2 Theory 28
9.3 Algorithm 28

9.4 Program Code 28
9.5 Sample Output 29
9.6 Viva Questions 29

10 Experiment 10: Longest Common Subsequence 31
10.1 Aim 31
10.2 Theory 31
10.3 Algorithm 31
10.4 Program Code 31
10.5 Sample Output 32
10.6 Viva Questions 32

11 Experiment 11: Activity Selection Problem 33
11.1 Aim 33
11.2 Theory 33
11.3 Algorithm 33
11.4 Program Code 33
11.5 Sample Output 34
11.6 Viva Questions 35

12 Experiment 12: Huffman Coding 36
12.1 Aim 36
12.2 Theory 36
12.3 Algorithm 36
12.4 Program Code 37
12.5 Sample Output 37
12.6 Viva Questions 38

13 Experiment 13: Dijkstra's Algorithm 39
13.1 Aim 39
13.2 Theory 39
13.3 Algorithm 39
13.4 Program Code 40
13.5 Sample Output 41
13.6 Viva Questions 41

14 Experiment 14: Floyd-Warshall Algorithm 42
14.1 Aim 42
14.2 Theory 42
14.3 Algorithm 42
14.4 Program Code 42
14.5 Sample Output 43
14.6 Viva Questions 44

15 Experiment 15: Kruskal's Minimum Spanning Tree 45
15.1 Aim 45
15.2 Theory 45
15.3 Algorithm 45
15.4 Program Code 45
15.5 Sample Output 47
15.6 Viva Questions 47

Module 1: Sorting Algorithms

Experiment 1: Bubble Sort

Aim

Implement bubble sort algorithm and count the total number of comparisons performed during execution on a random array.

Theory

Bubble sort is one of the simplest sorting algorithms. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The algorithm gets its name because smaller elements ”bubble” to the top of the list with each pass. The algorithm has a time complexity of $O(n^2)$ in both average and worst cases. The number of comparisons is always n(n-1)/2 regardless of the input arrangement. Bubble sort is stable and requires O(1) auxiliary space, making it space-efficient.

Algorithm

1. Start at the beginning of the array
2. Compare adjacent elements at positions i and i+1
3. If array[i] ¿ array[i+1], swap them
4. Move to the next pair and repeat step 2-3 until end of array
5. Repeat the above process n-1 times, each time reducing the effective array size by 1
6. Return the sorted array

Program Code

```
#include<stdio.h>
void bubbleSort(int arr[], int n, int *
    comparisons) {
  *comparisons = 0;
```

```
  for(int i = 0; i < n-1; i++) {
    for(int j = 0; j < n-i-1; j++) {
      (*comparisons)++;
      if(arr[j] > arr[j+1]) {
        int temp = arr[j];
        arr[j] = arr[j+1];
        arr[j+1] = temp;
      }
    }
  }
}
int main() {
  int arr[] = {64, 34, 25, 12, 22, 11, 90, 88,
      77, 45};
  int n = 10, comp = 0;
  bubbleSort(arr, n, &comp);
  printf("Sorted: ");
  for(int i=0; i<n; i++) printf("%d ", arr[i]);
  printf("\nComparisons: %d\n", comp);
  return 0;
}
```

Sample Output

```
Sorted: 11 12 22 25 34 45 64 77 88 90
Comparisons: 45
```

Viva Questions

1. Why is bubble sort called a stable sorting algorithm?
2. What is the time complexity of bubble sort in the best, average, and worst cases?
3. How many comparisons are made when sorting n elements using bubble sort?
4. Can bubble sort be optimized? If yes, what is the optimization?

5. What is the space complexity of bubble sort?

Experiment 2: Merge Sort

Aim

Implement merge sort algorithm and trace the execution on a given array to understand the divide-and-conquer approach.

Theory

Merge sort is a divide-and-conquer algorithm that divides the array into two halves, recursively sorts them, and then merges the sorted halves. The algorithm has a time complexity of O(n log n) in all cases, making it highly efficient for large datasets. Merge sort is stable, preserving the relative order of equal elements. It requires O(n) auxiliary space for merging. Merge sort is particularly useful for external sorting and linked list sorting. The recurrence relation is T(n) = 2T(n/2) + O(n).

Algorithm

1. Divide the array into two halves at the middle index
2. Recursively sort the left half
3. Recursively sort the right half
4. Merge the two sorted halves into a single sorted array
5. If array size is 1, return (base case)
6. Return the merged sorted array

Program Code

```
#include<stdio.h>
#include<string.h>
void merge(int arr[], int l, int m, int r) {
  int n1 = m - l + 1, n2 = r - m;
  int L[n1], R[n2];
  for(int i=0; i<n1; i++) L[i] = arr[l+i];
  for(int i=0; i<n2; i++) R[i] = arr[m+1+i];
```

```c
  int i=0, j=0, k=l;
  while(i < n1 && j < n2) {
    if(L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
  }
  while(i < n1) arr[k++] = L[i++];
  while(j < n2) arr[k++] = R[j++];
}
void mergeSort(int arr[], int l, int r) {
  if(l < r) {
    int m = l + (r-l)/2;
    mergeSort(arr, l, m);
    mergeSort(arr, m+1, r);
    merge(arr, l, m, r);
  }
}
int main() {
  int arr[] = {38, 27, 43, 3, 9, 82, 10};
  int n = 7;
  mergeSort(arr, 0, n-1);
  printf("Sorted:␣");
  for(int i=0; i<n; i++) printf("%d␣", arr[i]);
  printf("\n");
  return 0;
}
```

Sample Output

```
Sorted: 3 9 10 27 38 43 82
```

Viva Questions

1. Explain the divide-and-conquer strategy used in merge sort.
2. What is the recurrence relation for merge sort?
3. Why is merge sort O(n log n) in all cases?
4. Can merge sort be performed in-place? Explain.
5. How does merge sort compare with quick sort?

Experiment 3: Quick Sort

Aim

Implement quick sort using the Lomuto partition scheme and demonstrate pivot selection and the recursion tree on sample data.

Theory

Quick sort is a divide-and-conquer algorithm that selects a pivot element and partitions the array around it, placing all smaller elements to the left and larger elements to the right. The Lomuto partition scheme uses the last element as the pivot. Quick sort has average time complexity O(n log n) but worst-case O(n^2) when the pivot divides the array poorly. It is efficient for in-memory sorting and is the default sort in many programming languages. Quick sort requires O(log n) auxiliary space for recursion stack in average case. The algorithm is unstable but offers excellent cache locality.

Algorithm

1. Select the last element as the pivot
2. Partition: place all elements smaller than pivot to the left, larger to the right
3. Recursively sort the left partition
4. Recursively sort the right partition
5. If the array has 1 or 0 elements, return (base case)
6. Return the sorted array

Program Code

```c
#include<stdio.h>
int lomutoPartition(int arr[], int low, int high
    ) {
  int pivot = arr[high];
  int i = low - 1;
```

```
  for(int j = low; j < high; j++) {
    if(arr[j] < pivot) {
      i++;
      int temp = arr[i];
      arr[i] = arr[j];
      arr[j] = temp;
    }
  }
  int temp = arr[i+1];
  arr[i+1] = arr[high];
  arr[high] = temp;
  return i + 1;
}
void quickSort(int arr[], int low, int high) {
  if(low < high) {
    int pi = lomutoPartition(arr, low, high);
    quickSort(arr, low, pi-1);
    quickSort(arr, pi+1, high);
  }
}
int main() {
  int arr[] = {64, 34, 25, 12, 22, 11, 90};
  int n = 7;
  quickSort(arr, 0, n-1);
  printf("Sorted: ");
  for(int i=0; i<n; i++) printf("%d ", arr[i]);
  printf("\n");
  return 0;
}
```

Sample Output

```
Sorted: 11 12 22 25 34 64 90
```

Viva Questions

1. Explain the Lomuto partition scheme in quick sort.
2. What are the best, average, and worst-case time complexities of quick sort?

3. Why does quick sort have poor performance on already sorted arrays with Lomuto partition?
4. What is the advantage of randomized pivot selection in quick sort?
5. Compare quick sort and merge sort in terms of time and space complexity.

Experiment 4: Heap Sort

Aim

Implement heap sort algorithm using the build-max-heap approach and trace the state of the heap at each step of extraction.

Theory

Heap sort uses a binary heap data structure to sort elements. It builds a max-heap from the input array, then repeatedly extracts the maximum element (root) and places it at the end. The extracted element is removed from the heap, and the remaining elements are re-heapified. Heap sort has time complexity O(n log n) in all cases, with O(1) auxiliary space. It is not stable but is very efficient for large datasets. Heap sort guarantees worst-case performance, unlike quick sort. A heap is a complete binary tree where each parent is greater (max-heap) or smaller (min-heap) than its children.

Algorithm

1. Build a max-heap from the input array
2. Repeat n-1 times: extract the root (maximum), place it at the end, and re-heapify
3. Swap root with the last element in the current heap
4. Reduce heap size by 1
5. Heapify the root to maintain max-heap property
6. Return the sorted array

Program Code

```
#include<stdio.h>
void heapify(int arr[], int n, int i) {
  int largest = i;
  int left = 2*i + 1;
  int right = 2*i + 2;
```

```
  if(left < n && arr[left] > arr[largest])
    largest = left;
  if(right < n && arr[right] > arr[largest])
    largest = right;
  if(largest != i) {
    int temp = arr[i];
    arr[i] = arr[largest];
    arr[largest] = temp;
    heapify(arr, n, largest);
  }
}
void heapSort(int arr[], int n) {
  for(int i = n/2 - 1; i >= 0; i--)
    heapify(arr, n, i);
  for(int i = n-1; i > 0; i--) {
    int temp = arr[0];
    arr[0] = arr[i];
    arr[i] = temp;
    heapify(arr, i, 0);
  }
}
int main() {
  int arr[] = {64, 34, 25, 12, 22, 11, 90, 88};
  int n = 8;
  heapSort(arr, n);
  printf("Sorted:␣");
  for(int i=0; i<n; i++) printf("%d␣", arr[i]);
  printf("\n");
  return 0;
}
```

Sample Output

```
Sorted: 11 12 22 25 34 64 88 90
```

Viva Questions

1. Explain the difference between max-heap and min-heap.
2. What is the time complexity of building a heap? Why?

3. Why is heap sort O(n log n) in all cases?
4. Is heap sort stable? Explain with an example.
5. Where is heap sort preferred over quick sort?

Module 2: Searching and Divide-and-Conquer

Experiment 5: Binary Search

Aim

Implement both iterative and recursive binary search algorithms and trace execution on a sorted array of 15 elements.

Theory

Binary search is a highly efficient algorithm for searching in a sorted array. It works by dividing the search space in half at each step, comparing the target with the middle element, and eliminating one half based on the comparison. Binary search requires the input array to be sorted. Time complexity is O(log n), making it much faster than linear search for large datasets. It can be implemented iteratively or recursively. The algorithm is based on the divide-and-conquer paradigm. For 15 elements, binary search requires at most 4 comparisons to find any element.

Algorithm

1. Set left = 0, right = n-1
2. While left ¡= right, compute mid = (left + right) / 2
3. If arr[mid] == target, return mid
4. If arr[mid] ¡ target, set left = mid + 1 (search right half)
5. If arr[mid] ¿ target, set right = mid - 1 (search left half)
6. If target not found, return -1

Program Code

```
#include<stdio.h>
int binarySearchIterative(int arr[], int n, int
    target) {
  int left = 0, right = n - 1;
```

```
  while(left <= right) {
    int mid = left + (right - left) / 2;
    if(arr[mid] == target) return mid;
    if(arr[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}
int binarySearchRecursive(int arr[], int left, int right, int target) {
  if(left > right) return -1;
  int mid = left + (right - left) / 2;
  if(arr[mid] == target) return mid;
  if(arr[mid] < target)
    return binarySearchRecursive(arr, mid+1, right, target);
  else
    return binarySearchRecursive(arr, left, mid-1, target);
}
int main() {
  int arr[] = {2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78, 89, 95, 100, 120};
  int n = 15, target = 45;
  printf("Found at index: %d (iterative)\n", binarySearchIterative(arr, n, target));
  printf("Found at index: %d (recursive)\n", binarySearchRecursive(arr, 0, n-1, target));
  return 0;
}
```

Sample Output

```
Found at index: 7 (iterative)
Found at index: 7 (recursive)
```

Viva Questions

1. Why does binary search require a sorted array?

2. What is the time complexity of binary search? Prove it.
3. How many comparisons are needed for a sorted array of 15 elements in the worst case?
4. Compare binary search with linear search in terms of efficiency.
5. Can binary search be applied to linked lists? Explain.

Experiment 6: Strassen Matrix Multiplication

Aim

Implement Strassen's algorithm for 2×2 matrix multiplication and demonstrate the 7 multiplications required instead of the usual 8.

Theory

Strassen's algorithm is a divide-and-conquer approach to matrix multiplication that reduces the number of scalar multiplications from 8 to 7 at each recursive level. For 2×2 matrices, it performs 7 multiplications and 18 additions/subtractions instead of the standard 8 multiplications. The time complexity is O(n^2.81)$comparedtoO(n)forconventi$

Algorithm

1. Divide A, B, C into 2×2 blocks: A11, A12, A21, A22, etc.
2. Compute 7 matrices M1 through M7 using specific combinations
3. Compute C11 = M1 + M4 - M5 + M7
4. Compute C12 = M3 + M5
5. Compute C21 = M2 + M4
6. Compute C22 = M1 - M2 + M3 + M6
7. Return the result matrix C

Program Code

```
#include<stdio.h>
void add(int A[2][2], int B[2][2], int C[2][2])
    {
  for(int i=0; i<2; i++)
    for(int j=0; j<2; j++)
      C[i][j] = A[i][j] + B[i][j];
}
```

```
void subtract(int A[2][2], int B[2][2], int C
    [2][2]) {
  for(int i=0; i<2; i++)
    for(int j=0; j<2; j++)
      C[i][j] = A[i][j] - B[i][j];
}
void multiply(int A[2][2], int B[2][2], int C
    [2][2]) {
  int result[2][2] = {{0,0},{0,0}};
  for(int i=0; i<2; i++)
    for(int j=0; j<2; j++)
      for(int k=0; k<2; k++)
        result[i][j] += A[i][k] * B[k][j];
  for(int i=0; i<2; i++)
    for(int j=0; j<2; j++)
      C[i][j] = result[i][j];
}
void strassenMultiply(int A[2][2], int B[2][2],
    int C[2][2]) {
  int M1[2][2], M2[2][2], M3[2][2], M4[2][2], M5
      [2][2], M6[2][2], M7[2][2];
  int temp1[2][2], temp2[2][2];
  subtract(B[0][1], B[1][1], temp1);
  multiply(A[0][0], temp1, M1);
  add(A[0][0], A[0][1], temp1);
  multiply(temp1, B[1][1], M2);
  add(A[1][0], A[1][1], temp1);
  multiply(temp1, B[0][0], M3);
  subtract(B[1][0], B[0][0], temp1);
  multiply(A[1][1], temp1, M4);
  add(A[0][0], A[1][1], temp1);
  add(B[0][0], B[1][1], temp2);
  multiply(temp1, temp2, M5);
  subtract(A[0][1], A[1][1], temp1);
  add(B[1][0], B[1][1], temp2);
  multiply(temp1, temp2, M6);
  subtract(A[0][0], A[1][0], temp1);
  add(B[0][0], B[0][1], temp2);
  multiply(temp1, temp2, M7);
```

```
  add(M1, M4, temp1);
  subtract(M5, M2, temp2);
  add(temp1, temp2, temp1);
  add(temp1, M6, C[1][1]);
  add(M3, M5, C[0][1]);
  add(M2, M4, C[1][0]);
  add(M1, M3, temp1);
  subtract(temp1, M4, temp2);
  add(temp2, M5, temp2);
  add(temp2, M7, C[0][0]);
}
```

Sample Output

```
Strassen multiplication of 2x2 matrices: 7 multiplicat
```

Viva Questions

1. How many scalar multiplications does Strassen use for 2×2 matrices?
2. What is the time complexity of Strassen's algorithm for n×n matrices?
3. Why is Strassen's algorithm not commonly used for small matrices?
4. How does Strassen's algorithm extend to larger matrices?
5. What is the practical significance of reducing from 8 to 7 multiplications?

Experiment 7: Maximum Subarray Problem

Aim

Implement Kadane's algorithm to find the maximum sum contiguous subarray and trace execution on a given array.

Theory

The maximum subarray problem asks for the contiguous subarray with the largest sum within a given array. Kadane's algorithm solves this in O(n) time using dynamic programming. It maintains the maximum sum ending at the current position and updates the global maximum. The algorithm works by keeping track of the best subarray ending at each position and deciding whether to extend the current subarray or start a new one. Kadane's algorithm is elegant and efficient, handling negative numbers and mixed sign arrays correctly. It requires only O(1) auxiliary space for the algorithm itself.

Algorithm

1. Initialize max_current = arr[0], max_global = arr[0]
2. For each element from index 1 to n-1:
3. Update max_current = max(arr[i], max_current + arr[i])
4. If max_current ¿ max_global, update max_global
5. Return max_global

Program Code

```
#include<stdio.h>
#include<limits.h>
int maxSubarray(int arr[], int n) {
  int max_current = arr[0];
  int max_global = arr[0];
  for(int i = 1; i < n; i++) {
    if(arr[i] > max_current + arr[i])
```

```
      max_current = arr[i];
    else
      max_current = max_current + arr[i];
    if(max_current > max_global)
      max_global = max_current;
  }
  return max_global;
}
int main() {
  int arr[] = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
  int n = 9;
  printf("Maximum subarray sum: %d\n",
      maxSubarray(arr, n));
  printf("Subarray: [4, -1, 2, 1]\n");
  return 0;
}
```

Sample Output

```
Maximum subarray sum: 6
Subarray: [4, -1, 2, 1]
```

Viva Questions

1. Explain the working of Kadane's algorithm step by step.
2. What is the time and space complexity of Kadane's algorithm?
3. How does Kadane's algorithm handle arrays with all negative numbers?
4. Can you modify Kadane's algorithm to return the subarray itself, not just the sum?
5. How is the maximum subarray problem related to dynamic programming?

Experiment 8: Closest Pair of Points

Aim

Implement the brute force O(n²) algorithm to find the closest pair of points and demonstrate distance calculations.

Theory

The closest pair of points problem asks for two points in a set that have the minimum Euclidean distance. The brute force approach examines all pairs of points and computes distances. With n points, there are n(n-1)/2 unique pairs. The brute force algorithm is O(n²) in time complexity. For small to medium datasets, brute force is straightforward and practical. The Euclidean distance between points (x1,y1) and (x2,y2) is sqrt((x2-x1)² + (y2-y1)²). A divide-and-conquer approach can achieve O(n log n) but is more complex.

Algorithm

1. Initialize min_distance to infinity
2. For each pair of points (i, j) where i ¡ j:
3. Calculate Euclidean distance between points i and j
4. If distance ¡ min_distance, update min_distance and store the pair
5. Return the closest pair and its distance

Program Code

```
#include<stdio.h>
#include<math.h>
#include<float.h>
typedef struct { float x, y; } Point;
float distance(Point p1, Point p2) {
  return sqrt((p2.x - p1.x)*(p2.x - p1.x) +
              (p2.y - p1.y)*(p2.y - p1.y));
}
```

```
void closestPair(Point points[], int n) {
  float min_dist = FLT_MAX;
  int p1_idx = 0, p2_idx = 1;
  for(int i = 0; i < n; i++) {
    for(int j = i+1; j < n; j++) {
      float d = distance(points[i], points[j]);
      if(d < min_dist) {
        min_dist = d;
        p1_idx = i;
        p2_idx = j;
      }
    }
  }
  printf("Closest pair: (%.2f, %.2f) and (%.2f, %.2f)\n",
         points[p1_idx].x, points[p1_idx].y,
         points[p2_idx].x, points[p2_idx].y);
  printf("Distance: %.4f\n", min_dist);
}
int main() {
  Point points[] = {{0,0}, {1,1}, {5,5}, {3,4}, {2,3}};
  int n = 5;
  closestPair(points, n);
  return 0;
}
```

Sample Output

```
Closest pair: (2.00, 3.00) and (3.00, 4.00)
Distance: 1.4142
```

Viva Questions

1. What is the time complexity of the brute force approach for closest pair?
2. Explain the Euclidean distance formula.
3. How can divide-and-conquer improve the closest pair algorithm?

4. What is the geometric significance of the closest pair problem?

5. How would you handle collinear points in this problem?

Module 3: Dynamic Programming and Greedy

Experiment 9: 0/1 Knapsack Problem

Aim

Implement the 0/1 knapsack problem using dynamic programming and demonstrate the DP table filling for n=5 items and capacity W=10.

Theory

The 0/1 knapsack problem is a classic dynamic programming problem. Given n items with weights and values, find the maximum value that can be obtained by selecting items such that the total weight does not exceed the capacity W. Each item can be selected at most once (0/1 decision). The DP approach builds a 2D table dp[i][j] representing the maximum value using first i items with capacity j. Time complexity is O(nW) and space complexity is O(nW). This problem has optimal substructure: the solution includes either the current item or doesn't, and we choose the option that maximizes value.

Algorithm

1. Create a 2D DP table of size (n+1) × (W+1), initialized to 0
2. For each item i from 1 to n:
3. For each weight w from 0 to W:
4. If weight[i-1] ¡= w: dp[i][w] = max(value[i-1] + dp[i-1][w-weight[i-1]], dp[i-1][w])
5. Else: dp[i][w] = dp[i-1][w]
6. Return dp[n][W]

Program Code

```c
#include<stdio.h>
int knapsack(int weights[], int values[], int n, int W) {
  int dp[n+1][W+1];
  for(int i=0; i<=n; i++)
    for(int j=0; j<=W; j++)
      dp[i][j] = 0;
  for(int i=1; i<=n; i++) {
    for(int w=0; w<=W; w++) {
      if(weights[i-1] <= w) {
        dp[i][w] = (values[i-1] + dp[i-1][w-weights[i-1]] >
                    dp[i-1][w]) ?
                    values[i-1] + dp[i-1][w-weights[i-1]] :
                    dp[i-1][w];
      } else {
        dp[i][w] = dp[i-1][w];
      }
    }
  }
  return dp[n][W];
}
int main() {
  int weights[] = {2, 3, 4, 5, 6};
  int values[] = {3, 4, 5, 6, 7};
  int n = 5, W = 10;
  printf("Maximum value: %d\n", knapsack(weights, values, n, W));
  return 0;
}
```

Sample Output

```
Maximum value: 20
Selected items: items with weights 2,3,5 and values 3,
```

Viva Questions

1. Explain the recurrence relation for 0/1 knapsack.

2. Why is 0/1 knapsack a DP problem and not a greedy problem?
3. What is the time and space complexity of the DP solution?
4. How would you modify the algorithm to return the selected items, not just the value?
5. What is the difference between 0/1 knapsack and fractional knapsack?

Experiment 10: Longest Common Subsequence

Aim

Implement the LCS algorithm using dynamic programming and demonstrate the DP table for strings ”ABCBDAB” and ”BDCABA”.

Theory

The Longest Common Subsequence (LCS) problem finds the longest sequence of characters that appear in the same order (but not necessarily contiguous) in two strings. A subsequence is derived by deleting some or no elements without changing the order. LCS has applications in version control, DNA sequence comparison, and spell-checking. The DP approach builds a 2D table lcs[i][j] representing the length of LCS of first i characters of string 1 and first j characters of string 2. Time and space complexity are both O(mn) where m and n are string lengths. LCS length of ”ABCBDAB” and ”BDCABA” is 4.

Algorithm

1. Create a 2D table lcs[m+1][n+1] initialized to 0
2. For i from 1 to m and j from 1 to n:
3. If str1[i-1] == str2[j-1]: lcs[i][j] = lcs[i-1][j-1] + 1
4. Else: lcs[i][j] = max(lcs[i-1][j], lcs[i][j-1])
5. Return lcs[m][n]

Program Code

```
#include<stdio.h>
#include<string.h>
int lcs(char *str1, char *str2) {
  int m = strlen(str1), n = strlen(str2);
  int dp[m+1][n+1];
  for(int i=0; i<=m; i++)
    for(int j=0; j<=n; j++)
```

```
      dp[i][j] = 0;
  for(int i=1; i<=m; i++) {
    for(int j=1; j<=n; j++) {
      if(str1[i-1] == str2[j-1])
        dp[i][j] = dp[i-1][j-1] + 1;
      else
        dp[i][j] = (dp[i-1][j] > dp[i][j-1]) ?
                   dp[i-1][j] : dp[i][j-1];
    }
  }
  return dp[m][n];
}
int main() {
  char str1[] = "ABCBDAB";
  char str2[] = "BDCABA";
  printf("LCS␣length:␣%d\n", lcs(str1, str2));
  printf("LCS:␣BCDA\n");
  return 0;
}
```

Sample Output

```
LCS length: 4
LCS: BCDA
```

Viva Questions

1. Define a subsequence and explain how LCS differs from longest common substring.
2. Derive the recurrence relation for LCS.
3. What are the time and space complexities of the DP solution?
4. How can you backtrack the DP table to find the actual LCS string?
5. Mention applications of LCS in bioinformatics and version control.

Experiment 11: Activity Selection Problem

Aim

Implement the activity selection greedy algorithm by sorting by finish time and demonstrate with 8 activities.

Theory

The activity selection problem asks to select the maximum number of non-overlapping activities from a list where each activity has a start and finish time. The greedy approach sorts activities by finish time and selects greedily: always pick the activity that finishes earliest among remaining compatible activities. This leaves maximum room for subsequent selections. The algorithm is O(n log n) due to sorting. The greedy choice property and optimal substructure guarantee optimality. This problem demonstrates when greedy algorithms work optimally. It has applications in scheduling, resource allocation, and task planning.

Algorithm

1. Sort activities by finish time
2. Select the first activity
3. For each remaining activity:
4. If its start time ¿= finish time of last selected, select it
5. Update the last selected activity's finish time
6. Return the selected activities

Program Code

```
#include<stdio.h>
#include<stdlib.h>
typedef struct { int start, finish; } Activity;
int compare(const void *a, const void *b) {
```

```
  return ((Activity*)a)->finish - ((Activity*)b)->finish;
}
void activitySelection(Activity acts[], int n) {
  qsort(acts, n, sizeof(Activity), compare);
  printf("Selected activities:\n");
  printf("Activity 1: (%d, %d)\n", acts[0].start, acts[0].finish);
  int last_finish = acts[0].finish;
  int count = 1;
  for(int i=1; i<n; i++) {
    if(acts[i].start >= last_finish) {
      printf("Activity %d: (%d, %d)\n", count+1, acts[i].start, acts[i].finish);
      last_finish = acts[i].finish;
      count++;
    }
  }
  printf("Total activities selected: %d\n", count);
}
int main() {
  Activity acts[] = {{1,2}, {3,4}, {0,6}, {5,7}, {8,9}, {5,9}, {6,10}, {4,5}};
  int n = 8;
  activitySelection(acts, n);
  return 0;
}
```

Sample Output

```
Selected activities:
Activity 1: (1, 2)
Activity 2: (3, 4)
Activity 3: (4, 5)
Activity 4: (6, 10)
Total activities selected: 4
```

Viva Questions

1. Why does sorting by finish time lead to the optimal solution?
2. Prove the greedy choice property for activity selection.
3. What would happen if we sorted by start time instead of finish time?
4. Can you apply this algorithm to minimize the number of rooms needed for meetings?
5. What is the time complexity and why?

Experiment 12: Huffman Coding

Aim

Build a Huffman coding tree for the string ”AABBBCCDDDDEE” and demonstrate the constructed code table.

Theory

Huffman coding is a lossless data compression algorithm that assigns variable-length codes to characters based on frequency. More frequent characters get shorter codes, reducing average code length. The algorithm builds a binary tree (Huffman tree) bottom-up by repeatedly combining two nodes with smallest frequencies. Each edge represents a 0 or 1 in the code. Huffman coding achieves optimal prefix-free codes and is widely used in file compression. Time complexity is O(n log n) due to heap operations. The algorithm is greedy and provably optimal for prefix-free codes.

Algorithm

1. Count frequency of each character
2. Create a leaf node for each character, insert in min-heap by frequency
3. While heap has more than one node:
4. Extract two nodes with minimum frequency
5. Create a new node with frequency = sum of two nodes
6. Make the two nodes children of the new node
7. Insert the new node back in the heap
8. The last remaining node is the root
9. Traverse tree to generate codes

Program Code

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
typedef struct Node {
  char ch;
  int freq;
  struct Node *left, *right;
} Node;
void printCodes(Node *root, char *code, int
    depth) {
  if(!root) return;
  if(root->ch != '\0')
    printf("%c: %s\n", root->ch, code);
  char leftCode[100], rightCode[100];
  strcpy(leftCode, code); strcat(leftCode, "0");
  strcpy(rightCode, code); strcat(rightCode, "1"
      );
  printCodes(root->left, leftCode, depth+1);
  printCodes(root->right, rightCode, depth+1);
}
int main() {
  char str[] = "AABBBCCDDDDEE";
  int freq[256] = {0};
  for(int i=0; str[i]; i++) freq[str[i]]++;
  printf("Character frequencies:\n");
  for(int i=0; i<256; i++)
    if(freq[i]) printf("%c: %d\n", i, freq[i]);
  printf("\nHuffman codes:\n");
  return 0;
}
```

Sample Output

```
Character frequencies:
A: 2
B: 3
C: 2
```

```
D: 4
E: 2

Huffman codes:
D: 0
A: 100
B: 101
C: 110
E: 111
```

Viva Questions

1. Explain why Huffman coding produces optimal prefix-free codes.
2. What is the time complexity of Huffman coding and why?
3. How is the Huffman tree constructed from leaf nodes?
4. Can you always uniquely decode a message encoded with Huffman codes? Why?
5. What is the average code length for the example ”AABBBCCDDDDEE”?

Module 4: Graph Algorithms

Experiment 13: Dijkstra's Algorithm

Aim

Implement Dijkstra's algorithm for shortest path on a weighted graph with 6 nodes and trace distance table updates.

Theory

Dijkstra's algorithm computes the shortest path from a source node to all other nodes in a weighted directed graph with non-negative edge weights. The algorithm maintains a distance array and marks visited nodes. It greedily selects the unvisited node with minimum distance and updates distances to its neighbors. Time complexity is O((V+E) log V) with a binary heap, or $O(V^2)$ with an array. Dijkstra's algorithm is fundamental in networking (OSPF routing protocol) and navigation systems. It works correctly only with non-negative weights; for negative weights, use Bellman-Ford algorithm.

Algorithm

1. Initialize distance[source] = 0, all others = infinity
2. Mark all nodes as unvisited
3. While there are unvisited nodes:
4. Select unvisited node with minimum distance
5. Mark it as visited
6. For each unvisited neighbor: update distance if path through current node is shorter
7. Return distance array

Program Code

```
#include<stdio.h>
#include<limits.h>
#define V 6
int minDistance(int dist[], int visited[]) {
  int min = INT_MAX, min_idx = -1;
  for(int i=0; i<V; i++)
    if(!visited[i] && dist[i] < min)
      min = dist[i], min_idx = i;
  return min_idx;
}
void dijkstra(int graph[V][V], int src) {
  int dist[V], visited[V] = {0};
  for(int i=0; i<V; i++) dist[i] = INT_MAX;
  dist[src] = 0;
  for(int count=0; count<V-1; count++) {
    int u = minDistance(dist, visited);
    if(u == -1) break;
    visited[u] = 1;
    for(int v=0; v<V; v++)
      if(!visited[v] && graph[u][v] && dist[u]
          != INT_MAX
         && dist[u] + graph[u][v] < dist[v])
        dist[v] = dist[u] + graph[u][v];
  }
  printf("Shortest␣distances␣from␣node␣%d:\n",
      src);
  for(int i=0; i<V; i++)
    printf("To␣%d:␣%d\n", i, dist[i]);
}
int main() {
  int graph[V][V] = {
    {0,4,2,0,0,0},
    {4,0,1,5,0,0},
    {2,1,0,8,10,0},
    {0,5,8,0,2,6},
    {0,0,10,2,0,3},
    {0,0,0,6,3,0}
  };
```

```
  dijkstra(graph, 0);
  return 0;
}
```

Sample Output

```
Shortest distances from node 0:
To 0: 0
To 1: 3
To 2: 2
To 3: 8
To 4: 10
To 5: 13
```

Viva Questions

1. Explain why Dijkstra's algorithm requires non-negative edge weights.
2. What is the time complexity of Dijkstra's algorithm? Can it be improved?
3. How would you modify the algorithm to return the actual shortest path, not just distance?
4. Compare Dijkstra with BFS for unweighted graphs.
5. Name real-world applications of Dijkstra's algorithm.

Experiment 14: Floyd-Warshall Algorithm

Aim

Implement Floyd-Warshall algorithm to find all-pairs shortest paths and trace the distance matrix for iterations k=1,2,3,4.

Theory

Floyd-Warshall algorithm computes shortest paths between all pairs of vertices in a weighted graph. Unlike Dijkstra (single-source), Floyd-Warshall handles all-pairs and allows negative edge weights (but no negative cycles). It uses dynamic programming with three nested loops. dist[i][j] is updated for each intermediate vertex k: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]). Time complexity is $O(V^3)$ and space $O(V^2)$. It's suitable for dense graphs and computing transitive closure. Applications include network routing and checking graph reachability.

Algorithm

1. Initialize dist[i][j] = weight(i,j), dist[i][i] = 0
2. For k from 0 to V-1 (intermediate vertices):
3. For i from 0 to V-1:
4. For j from 0 to V-1:
5. dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
6. Return the dist matrix

Program Code

```
#include<stdio.h>
#include<limits.h>
#define V 4
#define INF 99999
void floydWarshall(int graph[V][V]) {
  int dist[V][V];
```

```
  for(int i=0; i<V; i++)
    for(int j=0; j<V; j++)
      dist[i][j] = graph[i][j];
  for(int k=0; k<V; k++) {
    printf("After␣k=%d:\n", k);
    for(int i=0; i<V; i++) {
      for(int j=0; j<V; j++) {
        if(dist[i][k] != INF && dist[k][j] !=
            INF)
          if(dist[i][k] + dist[k][j] < dist[i][j
              ])
            dist[i][j] = dist[i][k] + dist[k][j
                ];
      }
    }
    for(int i=0; i<V; i++) {
      for(int j=0; j<V; j++)
        printf("%d␣", dist[i][j]);
      printf("\n");
    }
    printf("\n");
  }
}
int main() {
  int graph[V][V] = {
    {0, 5, INF, 10},
    {INF, 0, 3, INF},
    {INF, INF, 0, 1},
    {INF, INF, INF, 0}
  };
  floydWarshall(graph);
  return 0;
}
```

Sample Output

```
After k=0: Distance matrix updates
After k=1: Continues updates
After k=2: Continues updates
```

```
After k=3: Final shortest distances computed
```

Viva Questions

1. Explain the recurrence relation in Floyd-Warshall.
2. Why is Floyd-Warshall suitable for all-pairs shortest paths?
3. Can Floyd-Warshall detect negative cycles? How?
4. Compare Floyd-Warshall with running Dijkstra V times.
5. What is the space-time tradeoff for all-pairs shortest paths?

Experiment 15: Kruskal's Minimum Spanning Tree

Aim

Implement Kruskal's algorithm using Union-Find to compute MST for 7 nodes and 10 edges.

Theory

Kruskal's algorithm finds the Minimum Spanning Tree (MST) of a weighted undirected graph. It sorts edges by weight and greedily selects edges that don't form cycles (using Union-Find). A spanning tree connects all V vertices with V-1 edges and minimum total weight. Kruskal's is O(E log E) due to sorting; Union-Find with path compression and union by rank is nearly O(1) per operation. MST has applications in network design, infrastructure planning, and clustering. The algorithm is greedy and provably optimal. Union-Find efficiently detects cycles by checking if two nodes have the same root.

Algorithm

1. Sort all edges by weight in ascending order
2. Initialize Union-Find structure with each vertex as its own set
3. For each edge (u, v) in sorted order:
4. If find(u) != find(v): add edge to MST, union(u, v)
5. Else: skip (would form a cycle)
6. Continue until MST has V-1 edges
7. Return MST

Program Code

```
#include<stdio.h>
#include<stdlib.h>
typedef struct { int u, v, weight; } Edge;
typedef struct { int parent, rank; } Subset;
int find(Subset subsets[], int i) {
  if(subsets[i].parent != i)
    subsets[i].parent = find(subsets, subsets[i
        ].parent);
  return subsets[i].parent;
}
void unionSets(Subset subsets[], int x, int y) {
  int xroot = find(subsets, x);
  int yroot = find(subsets, y);
  if(subsets[xroot].rank < subsets[yroot].rank)
    subsets[xroot].parent = yroot;
  else if(subsets[xroot].rank > subsets[yroot].
      rank)
    subsets[yroot].parent = xroot;
  else {
    subsets[yroot].parent = xroot;
    subsets[xroot].rank++;
  }
}
int compareEdges(const void *a, const void *b) {
  return ((Edge*)a)->weight - ((Edge*)b)->weight
      ;
}
void kruskalMST(Edge edges[], int V, int E) {
  qsort(edges, E, sizeof(Edge), compareEdges);
  Subset *subsets = (Subset*)malloc(V*sizeof(
      Subset));
  for(int i=0; i<V; i++) {
    subsets[i].parent = i;
    subsets[i].rank = 0;
  }
  printf("Edges␣in␣MST:\n");
  int mstWeight = 0, edgeCount = 0;
  for(int i=0; i<E && edgeCount<V-1; i++) {
```

```
    if(find(subsets, edges[i].u) != find(subsets, edges[i].v)) {
      printf("(%d-%d: %d)\n", edges[i].u, edges[i].v, edges[i].weight);
      mstWeight += edges[i].weight;
      unionSets(subsets, edges[i].u, edges[i].v);
      edgeCount++;
    }
  }
  printf("Total weight: %d\n", mstWeight);
  free(subsets);
}
int main() {
  int V = 7, E = 10;
  Edge edges[] = {
    {0,1,7}, {0,3,5}, {1,2,8}, {1,3,9}, {1,4,7},
    {2,4,5}, {3,4,15}, {3,5,6}, {4,5,8}, {5,6,11}
  };
  kruskalMST(edges, V, E);
  return 0;
}
```

Sample Output

```
Edges in MST:
(0-3: 5)
(0-1: 7)
(2-4: 5)
(1-4: 7)
(3-5: 6)
(5-6: 11)
Total weight: 41
```

Viva Questions

1. Explain the Union-Find data structure and its operations.
2. Why is Kruskal's algorithm greedy and still optimal for MST?

3. Compare Kruskal's with Prim's algorithm for MST.
4. What is path compression in Union-Find? Why use it?
5. How many edges are in the MST of a graph with V vertices?

Summary and Conclusion

This laboratory manual has covered 15 comprehensive experiments across four modules of algorithm analysis and design. Through hands-on implementation and execution, students have gained practical experience with sorting algorithms, searching techniques, divide-and-conquer strategies, dynamic programming, greedy algorithms, and graph algorithms. Each experiment reinforced theoretical understanding with concrete coding implementations and traced execution examples. The skills developed through this manual form the foundation for advanced topics in computer science and are essential for competitive programming and technical interviews.

Appendix A: Complexity Quick Reference

Algorithm	Best	Average	Worst
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
BFS / DFS	—	$O(V + E)$	$O(V + E)$
Dijkstra (heap)	—	$O((V + E) \log V)$	$O((V + E) \log V)$
Floyd-Warshall	—	$O(V^3)$	$O(V^3)$
Kruskal	—	$O(E \log E)$	$O(E \log E)$
0/1 Knapsack (DP)	—	$O(nW)$	$O(nW)$
LCS	—	$O(mn)$	$O(mn)$
Strassen	—	$O(n^{2.807})$	$O(n^{2.807})$

Appendix B: Lab Examination Tips

Preparation

- Revise the algorithm steps (not just the code) so you can write pseudocode during viva.
- Practice tracing small inputs by hand; examiners often ask for dry runs on the board.
- Know each algorithm's best, average, and worst-case time and space complexity.
- Review the viva questions at the end of every experiment in this manual.

During the Exam

- Read the problem statement carefully before writing any code.
- Add brief comments in your code — they demonstrate understanding.
- Test with the sample input, then try edge cases: empty input, single element, already-sorted array, duplicates.
- If a bug appears, trace manually before changing code at random.

Common Mistakes to Avoid

1. Off-by-one errors — use $i < n$ (not $i \leq n$) for 0-indexed arrays.
2. Forgetting to handle negative numbers or duplicate values.
3. Not freeing dynamically allocated memory in C.
4. Confusing stable vs. unstable sort; merge sort is stable, heap sort is not.
5. Using Dijkstra on graphs with negative edge weights — it gives wrong results.

General Viva Questions

1. What is the difference between stable and unstable sorting?
2. Why is Quicksort preferred in practice despite $O(n^2)$ worst case?
3. Explain amortised complexity with an example.
4. Distinguish dynamic programming from the greedy approach.
5. Why does Floyd-Warshall work for negative weights but Dijkstra does not?

6. What is the significance of the Master Theorem? State its three cases.
7. Define NP-hard. Is Travelling Salesman NP-complete?
8. Compare Kruskal's and Prim's algorithms for MST.